Recoverable Mutual Exclusion with Constant Amortized RMR Complexity from Standard Primitives

David Yu Cheng Chan
david.chan1@ucalgary.ca
University of Calgary
Calgary, Alberta, Canada

Philipp Woelfel
woelfel@ucalgary.ca
University of Calgary
Calgary, Alberta, Canada

ABSTRACT

Motivated by advances in non-volatile memory technology, recent research in mutual exclusion has focused on algorithms for a shared memory model, in which failed processes can recover from crashes. Golab and Ramaraju [9] defined the *recoverable mutual exclusion* problem, where a process may crash during a mutual exclusion protocol. Upon crashing a process's local memory is erased, and it starts a recovery procedure. The contents of the shared memory survives process failures.

The most efficient n-process mutual exclusion algorithms from standard synchronization primitives have an RMR complexity of $O(\log n/(\log\log n))$ [7, 11] per passage. Faster RMR complexity has only been achieved using artificially defined synchronization primitives, such as fetch-and-store-and-store, which are not available in actual systems [7, 10]. In this paper we present a mutual exclusion algorithm that uses only the standard primitives compare-and-swap and fetch-and-increment, which are available on common architectures, and achieves an amortized RMR complexity of $O(1)$ per passage in the DSM and the CC model.

CCS CONCEPTS

• **Theory of computation** → **Shared memory algorithms**; • **Software and its engineering** → *Mutual exclusion*; • **Computer systems organization** → Reliability.

KEYWORDS

recoverable mutual exclusion, asynchronous system, shared memory, fetch and increment

ACM Reference Format:
David Yu Cheng Chan and Philipp Woelfel. 2020. Recoverable Mutual Exclusion with Constant Amortized RMR Complexity from Standard Primitives. In *ACM Symposium on Principles of Distributed Computing (PODC '20), August 3–7, 2020, Virtual Event, Italy*. ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3382734.3405736

1 INTRODUCTION

Recent developments in memory architecture have led to the design of algorithms for a shared memory model in which processes can recover from failures. This model assumes that shared memory variables survive process crashes, for example because they are stored on non-volatile main memory (NVRAM). If a process crashes, it loses the contents of its local (volatile) memory, including the state of the process counter. It then enters a recovery procedure allows it to return to processing its pending tasks.

In that context, Golab and Ramaraju defined the *recoverable mutual exclusion* problem [9]. A mutual exclusion algorithm is a shared object that supports methods Enter() and Exit(), and calls to these methods must alternate, starting with Enter(). A process is in the *critical section* (CS) when it has finished an Enter() call but not yet called Exit(). It is in the *remainder section* (RS), if it is not in the CS and has no pending Enter() or Exit() call. Such algorithms satisfy at least two conditions: *Mutual exclusion* requires that no two processes are in the critical section at the same time, and *Deadlock-freedom* requires that some process with a pending Enter() or Exit() call will eventually finish this call, provided that all processes that are not in the RS keep taking steps.

A recoverable mutual exclusion algorithm provides an additional method, Recover(). If a process that is not in the RS crashes, all its local variables are reset to their initial values, and the process immediately begins executing method Recover(). In addition to mutual exclusion, a recoverable mutual exclusion algorithm must satisfy *critical section re-entry*, which means that if a process crashes in the CS, then it will reenter the CS before any other process. (It is the user's responsibility to design the code executed in the CS such that it tolerates processes reentering the CS multiple times in case of crashes.) Now *deadlock-freedom* only guarantees progress if, in addition to processes continuing to take steps, there are only finitely many crashes. (Otherwise a process might repeatedly crash in its CS, preventing other processes from making progress.) There are several other desirable properties that are satisfied by ours and other recoverable mutual exclusion algorithms, and we will describe them in the next section.

The standard complexity measure for mutual exclusion algorithms is RMR complexity. There are two common models for the analysis. In the cache-coherent (CC) model, processes are equipped with caches, and whenever a process performs a read operation it obtains a cache-copy of the corresponding memory location. Any non-read operation of that memory location invalidates all cache copies. A process's operation incurs a *remote memory reference* (RMR), if it is a read operation and the process has no valid cache copy, or if it is not a read operation. In the distributed shared memory (DSM) model, the shared memory is partitioned into segments, one for each process. An operation on a shared memory location incurs an RMR if and only if that memory location is not in the calling process's memory segment. Almost all work on the efficiency

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
PODC '20, August 3–7, 2020, Virtual Event, Italy
© 2020 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-7582-5/20/08…$15.00
https://doi.org/10.1145/3382734.3405736

of (recoverable and non-recoverable) mutual exclusion algorithms has considered RMR complexity in one of those two models.

A *passage* of a process begins when it calls Enter() and ends when the process crashes or when it finishes its following Exit() call. A *super-passage* of a process begins when the process calls Enter() and when it completes its following Exit() call. (Note that in the absence of process crashes, all passages and super-passages coincide.) The RMR complexity of a mutual exclusion algorithm is the maximum number of RMRs a process may incur in any passage. In the *non-recoverable* model, $\Theta(\log n)$ is a tight bound for the RMR complexity of mutual exclusion algorithms using only (read/write) registers [2, 16]. Compare-and-swap (CAS) objects can be implemented from registers with constant RMR complexity [6], so they do not help. Using fetch-and-store (FAS) or other standard synchronization primitives, it is possible to obtain constant RMR complexity [3, 5, 13].

Golab and Ramaraju [9] presented the first *recoverable* mutual exclusion algorithm. It uses FAS and CAS objects and achieves an RMR complexity of $O(\log n)$ in the CC model. Algorithms with RMR complexity $O(\log n/\log\log n)$ were devised by Golab and Hendler [7] for the CC model, and for the CC and DSM model by Jayanti, Jayanti, and Joshi [11]. Dhoked and Mittal improved the performance for executions where the number of process crashes is small [4]. Their algorithm has RMR complexity $O(\min\{1+\sqrt{f}, \log n/\log\log n\})$, where f is the total number of crashes in the "recent" past. Unlike in the non-recoverable model, prior to this work no recoverable algorithm was known that uses standard synchronization primitives and achieves $o(\log n/(\log\log n))$ RMR complexity. To fathom system assumptions that allow recoverable mutual exclusion with constant RMR complexity, esoteric synchronization primitives were considered, such as fetch-and-store-and-store and double word CAS [7, 10]. Current hardware architectures do not support these synchronization primitives. Golab and Hendler [8] also considered a different failure model, where all processes crash simultaneously.

In this paper we present the first recoverable mutual exclusion algorithm that achieves constant amortized RMR complexity per passage, and which uses, in addition to registers, only the standard synchronization primitives fetch-and-increment (FAI) and compare-and-swap (CAS). These are supported by common hardware architectures. Our algorithm achieves the desired RMR complexity on both the CC and the DSM model.

Theorem 1. *There is a recoverable mutual exclusion algorithm from FAI and CAS objects, such that in any execution in which m passages begin, the total number of RMRs is bounded by $O(m)$ on the CC and the DSM model.*

Our algorithm uses a completely different approach than previous recoverable mutual exclusion algorithms, which rely on FAS objects. (Recall that a FAS object supports a FAS(x) operation, which writes x to the object and returns the object's previous value.) Those algorithms rely on a common concept: Each process p performs a FAS(p) operation on a FAS object, and as a result obtains the ID of the previous process performing its FAS() operation on the same object. This allows processes to form a queue, and thus to coordinate their entry into the critical section. But if a process p crashes immediately after its FAS(p) operations, it overwrites a potential predecessor without getting any information. In particular, the information which process is a predecessor of p (or whether there is any predecessor) may never be recovered.

This inspires the idea of using objects, where information is not all lost in case of a process crash. FAI has that property: If a process crashes after performing a FAI() operation, the object's state still reflects the total number FAI() operations performed on it.

Our algorithm uses two unbounded FAI objects and an unbounded number of CAS objects. It is straight forward to bound the values stored in the FAI object, and also the number of CAS objects without sacrificing amortized RMR complexity, if the system provides memory management (e.g., a garbage collector). If memory management is not available, then we believe that standard techniques for memory reclamation [1, 14] may be used in the context of recoverable mutual exclusion. However, doing so is beyond the scope of this work and would distract from the core ideas of our algorithm.

In contrast to, e.g., [4, 7, 11], our algorithm does not provide any worst-case bounds on the RMRs a single process may need per passage. However, it is easy to see that if there are f process crashes, then each process performs at most $O(1+f)$ RMRs per passage. In particular we have constant worst-case RMR complexity in the absence of process crashes.

2 PRELIMINARIES

In the recoverable model, progress conditions can be defined in a similar way as in the non-recoverable model; the main difference is that processes may not make progress if they crash. We call a method *wait-free*, if each process either finishes its call to that method within a finite number of its own steps, or if it crashes during the method call. A method is *deadlock-free*, if a pending method call will terminate provided that there are only finitely many crashes, and all processes that have pending method calls or that are in the critical section keep taking steps. *Starvation-freedom*[1] is defined in the same way, except that now *all* pending method calls must terminate eventually (under the same conditions).

The Recover() method of our algorithm is wait-free, and we assume that it is called if and only if a process crashes while it is not in the remainder section.

The minimum guarantees we would expect from a recoverable mutual exclusion algorithm are mutual exclusion, critical section re-entry, and deadlock-freedom. Recall that **mutual exclusion** means that at most one process is in the critical section at any time and **critical section re-entry** requires that if a process p crashes in the critical section, then no other process enters the critical section during the interval starting from p's crash and until p re-enters the critical section. **Deadlock-freedom** means that all methods of the mutual exclusion algorithm are deadlock-free.

Similarly as the algorithms in [9, 11], our recoverable mutual exclusion algorithm satisfies several other desirable progress properties.

In particular, it satisfies **wait-free critical section re-entry**: There is a bound b such that, if a process crashes while in the CS, it reenters the CS before completing b consecutive steps without crashing. The algorithm also satisfies **wait-free recovery to exit**: There is a bound b such that if a process crashes after its first and

[1] What is called starvation-freedom in [9] was renamed to *weak* starvation-freedom in [11]. We chose to adhere to the original terminology.

before its last shared memory operation of Exit(), then it returns to Exit() before completing b consecutive steps without crashing. In addition, our algorithm satisfies **starvation-freedom**, meaning that all its methods are starvation-free.[2] The recoverable mutual exclusion algorithm of Jayanti, Jayanti and Joshi [11] satisfies a *strong starvation-freedom* property that our algorithm does not possess: In their algorithm all method calls are guaranteed to finish, provided that each process crashes only a finite number of times during each super-passage, even though there may be an infinite number of crashes in total.

Since our algorithm is not strong starvation free, it does also not guarantee fairness, such as **first-come first served** (FCFS). This property means that if some process p finishes a predefined wait-free prefix of the entry section (called doorway) before another process q begins its entry section, then p either crashes or enters the critical section before q. This property is satisfied by other algorithms with higher RMR complexity [12, 15]. Our algorithm satisfies FCFS only in the absence of process crashes.

It is also easy to make changes to our algorithm so that it satisfies **wait-free exit**, meaning that the Exit() method is wait-free. We explain how to achieve this in Section 3.4, as opposed to accommodating the pseudo-code, because the techniques to achieve are standard and do not contribute much in terms of new insights.

Our algorithm uses FAI and CAS objects. An FAI object stores an integer value that is initially 0. It supports only one operation, FAI(), which increments the stored integer by 1, and returns the previous value. A CAS object supports the operations read(), write(), and CAS(). read() and write() behave as expected. Operation CAS(*old*, *new*) compares the value of the object with *old*, and, if both match, replaces the object's value with *new* and returns **true**. Otherwise, it returns **false**. In our algorithm we use write() operations on CAS objects for convenience, but it is easy to see that they can be replaced with CAS() operations, should write() operations not be available.

3 THE RECOVERABLE MUTUAL EXCLUSION ALGORITHM

In this section we present our recoverable mutual exclusion algorithm. We first give an overview of the general idea. Then, in Section 3.2 we present a line by line description of the pseudo-code, which is given in Figure 1 on p. 6. After that, in Section 3.3 we prove the correctness of our algorithm. The analysis of RMR complexity is omitted for brevity. Finally, in Section 3.4 we explain how to achieve wait-free exit.

3.1 The General Idea

We first describe the main idea of our algorithm. The exact pseudocode may appear quite a bit different from this description, mainly because some engineering is needed to make all of this work. The aim of the following description is to convey the core insights that led to the algorithm design.

[2]Terminology in previous work differs slightly. E.g., in [9, 11] the definition of starvation-freedom applies only to the Enter() calls. Starvation-freedom of super-passages is then implied by other required progress conditions (e.g., *terminating* or *wait-free exit*). We find it most intuitive to call a mutual exclusion object deadlock- or starvation-free if and only if all its methods are deadlock- respectively starvation-free.

A simple approach for mutual exclusion in the non-recoverable model uses an FAI object C with initial value 0, and an infinite array $A[1..\infty]$ of CAS objects. In its entry protocol a process obtains a unique ticket number by performing an FAI() operation on C and adding one to the return value. The first process to enter the critical section is the one that received ticket 1. In its exit section, a process p that received a ticket with number w changes the value of $A[w+1]$ to notify the process that received (or will receive) ticket number $w+1$ that it is that process's turn to enter the critical section.

In the recovery model, this simple approach fails, because a process may crash immediately after incrementing C and getting its ticket number w. When recovering, the process does not know whether it incremented C at all, and as a result, there is no process trying to enter the CS using ticket w. This situation is very difficult to distinguish from one, where the process that received ticket w does not crash, but is only very slow.

We can deal with this as follows. Immediately after receiving a ticket w, a process p records in a dedicated shared memory location $W[p]$ the ticket number it received. Then it changes the value of $A[w]$ (using a CAS operation) to its process ID. We say the process *claims* its ticket w when it performs that CAS operation. Finally, the process increments another FAI object, D, to indicate that it has claimed its ticket. Note that if there are no process crashes, then the FAI object D will eventually catch up with C, and when that happens, every processes that received a ticket has claimed it.

When a process p recovers from a crash during one of these steps, it can check $W[p]$ to see if it has already recorded a ticket. If not, then it starts over by incrementing C and proceeding as above. Otherwise, if p has recorded a ticket, then instead it updates $A[w]$ accordingly (if necessary) and increments D, but not C. Therefore, each time a process crashes, it may increment D during its recovery procedure without incrementing C. We prove that D can be incremented only twice for each process crash, so we maintain the invariant that $D \leq C + 2F$, where F is the total number of crashes.

We also use a CAS object *Token*, which is initially 0. If there are no failures, then *Token* indicates the ticket of the last process that entered the critical section. With failures, some tickets will be skipped, and if there is no process in the critical section, then *Token* indicates the last ticket that was skipped.

Now suppose process q leaves the critical section. At that point $Token = w_q$, where w_q is the ticket q received in the entry section. If $D \leq C$, then process q increments *Token* and then checks if some process has already claimed $A[Token]$, in which case it notifies that process to enter the critical section. If no process has claimed $A[Token]$, then q "skips" the ticket with value *Token*, increments *Token* again, and checks the next position of $A[]$. It keeps doing so until $Token = C$ or until a process has claimed $A[Token]$.

There are two possible reasons that may lead to q skipping a ticket number w: First, the process p that received ticket w (as a result of its FAI on C) crashed immediately after receiving the ticket, and thus the ticket is lost. Second, process p does not crash after receiving ticket w, but is too slow to claim it. Using CAS operations on $A[w]$, processes q and p can agree whether p gets skipped or not, and if p gets skipped, it starts over the entry protocol by incrementing C again.

To see that this approach yields constant amortized RMR complexity, we observe that the total number of skipped tickets is at

most twice the number of process crashes. For example, if $F = 0$ (i.e., no processes crash), then *Token* will only be moved if $D = C$, and at that time it is guaranteed that all processes that received a ticket have already claimed their ticket. Similarly, for $F > 0$ there are at most $2F$ processes that received a ticket but have not yet claimed their ticket, when $D \geq C$.

A somewhat tricky part of the algorithm is to move *Token* and then possibly notify a process that claimed a ticket with value *Token* to enter the critical section. This needs to be done not only by a process leaving the critical section, but whenever some state of the data structure changed in such a way that there may be a process that has claimed a ticket and is now waiting to be notified. We use a method Move_Token() in which a process first checks whether it is permissible to move the *Token* (i.e., there is no process in the CS and $C \leq D$), then it moves the token, and finally it notifies a process to enter the CS, if appropriate. A process calls this method whenever it performed an action that may have made it necessary to move the token: After incrementing D, after leaving the CS, and after successfully moving the token.

3.2 Detailed Algorithm Description

The algorithm uses the following shared registers:

(1) $A[1..\infty]$: A queue of attempts to enter the critical section. For all $c \in \mathbb{Z}^+$, $A[c]$ stores either the ID of the process that makes the c-th attempt in the queue, or 0 if no process has made the c-th attempt, or -1 if the c-th attempt has been skipped. Initially $A[c] = 0$.
(2) C: A FAI object that stores the total number of times that a process has tried to join the queue $A[1..\infty]$ with an attempt to enter the critical section. Initially $C = 0$.
(3) D: A FAI object that stores a lower bound on the total number of times any process has joined (or rejoined) the queue $A[1..\infty]$ with an attempt to enter the critical section plus the total number of times any process has crashed. Note that a process that crashes may rejoin the queue $A[1..\infty]$ multiple times in the same position. Initially $D = 0$.
(4) $W[1..n]$: For every $p \in \{1, \ldots, n\}$, $W[p]$ stores (c, b) where c is latest position of the queue $A[1..\infty]$ that has been reserved for an attempt of p to enter the critical section (or 0 if no position is reserved) and b is whether p may enter the critical section. Initially $W[p] = (0, \textbf{false})$.
(5) $Exit[1..n]$: For every $p \in \{1, \ldots, n\}$, $Exit[p]$ stores whether p should be sent to Exit() if it crashes. Initially $Exit[p] = \textbf{false}$.
(6) *Token*: Stores an integer ℓ such that every attempt to enter the critical section within $A[1..\ell]$ has either succeeded or been skipped. Initially $Token = 0$.

During Enter() a process p performs the following steps:

(1) It checks whether it already has a ticket. If not, it takes a ticket number c from C via an FAI() operation (line 4). (The ticket number is the number that p set C to.) It then sets[3] $W[p]$ to $(c, \textbf{false})$, recording its ticket number in case it crashes later (line 5).
(2) It calls Waiting_Room() (line 6).
(3) If Waiting_Room() returns **true**, it enters the critical section (line 7); Otherwise it restarts Enter() from the beginning.

A process p performs the following steps in Waiting_Room():

(1) It reads $W[p]$ to find its ticket number w (line 23).
(2) It tries to join the queue $A[1..\infty]$ at slot $A[w]$ with a CAS$(0, p)$ (line 24).
(3) If it is unsuccessful in joining the queue (line 25), it writes $(0, \textbf{false})$ to $W[p]$ (line 26) to indicate that its previous ticket has become invalid, then returns **false** to get another ticket in Enter() (line 27).
(4) It increments D to increase the known lower bound on the total number of times any process has joined (or rejoined) the queue $A[1..\infty]$ with an attempt to enter the critical section plus the total number of times any process has crashed (line 28).
(5) It calls Move_Token() (line 29) to help move the queue $A[1..\infty]$ through the critical section.
(6) It waits until $W[p]$ contains $(w, \textbf{true})$, which indicates that p may now enter the critical section (lines 30-31).
(7) It returns **true** to Enter() (line 32), indicating that the wait is over, i.e., p may now enter the critical section.

Inside Move_Token(), a process p performs the following steps:

(1) It reads *Token* to find the number ℓ such that every attempt to enter the critical section within $A[1..\ell]$ has either succeeded or been skipped (line 34).
(2) It reads D and C (lines 35-36).
(3) It checks whether C contains an integer $c > \ell$ (line 37). If so, that indicates that at least one process wants to enter the critical section at this time, i.e., there could be attempts in $A[\ell + 1..\infty]$. So p reads $A[\ell + 1]$ to find the ID p' of the process that makes the attempt in $A[\ell + 1]$ (line 38).
(4) If $p' > 0$ (line 39), then p' is a process that makes an attempt to enter the critical section in $A[\ell + 1]$. So p tries to set $W[p']$ to $(\ell + 1, \textbf{true})$ so that p' may now enter the critical section (line 40). Then it checks whether p' is itself and whether $W[p']$ no longer contains $(\ell + 1, \textbf{true})$ (line 41), which would indicate that it no longer wants the critical section, so it should move the token (line 42). If it successfully moves the token, it calls Move_Token() again (line 43) to help move more of the queue $A[1..\infty]$ through the critical section.
(5) Otherwise if $p' \leq 0$, it checks whether the value c that it last read from C is at most d where d is the value it last read from D (line 45). Intuitively, if $c \leq d$, it is possible that the process who received the ticket $\ell + 1$ from C crashed and lost the ticket, so $A[\ell + 1]$ may never contain an attempt to enter the critical section. Thus p tries to skip over it by performing an CAS$(0, -1)$ on $A[\ell + 1]$ (line 46). If this CAS$(0, -1)$ is successful, $A[\ell + 1]$ is now blocked from ever containing an attempt to enter the critical section, so p moves *Token* forward to $\ell + 1$ (line 48) to skip over it. If it successfully moves the token, it calls Move_Token() again (line 49) to help move more of the queue $A[1..\infty]$ through the critical section.

Whenever a process p leaves the critical section to return to the remainder section, it performs the following steps in the Exit() procedure:

[3] This is achieved with a CAS operation, which, as we prove, always succeeds. Therefore, one could use a write operation instead, but using CAS simplifies our correctness proofs .

(1) It sets *Exit*[p] to **true** (line 9), to indicate that currently, it should be sent to Exit() if it crashes.
(2) It writes $(0, \textbf{false})$ to $W[p]$, indicating that its old ticket has expired (lines 10).
(3) It calls Move_Token() (line 11) to help move more of the queue $A[1..\infty]$ through the critical section.
(4) It sets *Exit*[p] to **false** (line 12), to indicate that currently, it should not be sent to Exit() if it crashes.
(5) It returns to the remainder section (line 13).

Whenever a process p recovers after a crash it performs the following steps:

(1) It increments D (line 15)to increase the known lower bound on the total number of times any process has joined (or rejoined) the queue $A[1..\infty]$ with an attempt to enter the critical section plus the total number of times any process has crashed.
(2) It reads *Exit*[p] (line 16). If it is **true**, then p jumps to Exit() (line 17).
(3) It reads $W[p]$ (line 18). If $W[p]$ contains $(-, \textbf{true})$, then p enters the critical section directly (lines 19-20).
(4) Otherwise, p jumps to Enter() (line 21).

3.3 Analysis

In this section we prove that the algorithm achieves the desired properties, summarized in the following theorem.

Theorem 2. *The recoverable mutual exclusion algorithm in Figure 1 satisfies mutual exclusion, deadlock-freedom, wait-free critical section re-entry, starvation freedom, wait-free recovery, and wait-free recovery to exit. The total number of RMRs incurred in any execution in which m passages begin is $O(m)$ on the DSM and the CC model.*

The worst-case RMR complexity is $O(1 + f)$ per passage for executions in which at most f process crashes occur in total. For executions with $f = 0$, first-come first-served (FCFS) is satisfied.

The remainder of this subsection is devoted to the proof of Theorem 2. For brevity, some proofs have been omitted. Let $\mathcal{E}$ be any run of the recoverable mutual exclusion algorithm in Figure 1.

Lemma 2.1. *The following properties must hold in this run $\mathcal{E}$:*

(1) *For every $p \in \{1, \ldots, n\}$, only p can perform operations on Exit[p].*
(2) *At any time t, for each $p \in \{1, \ldots, n\}$, if Exit[p] = **true**, then at the latest time in $[0, t]$ such that p is not in Recover(), p is in Exit().*
(3) *C, D, and Token are monotonically non-decreasing.*
(4) *At any time, Token contains some non-negative integer $\ell \in \{0, \ldots, C\}$.*
(5) *For every $p \in \{1, \ldots, n\}$ and every pair of integers w and w' such that $w \neq w'$, only process p can change $W[p]$ from $(w, -)$ to $(w', -)$.*
(6) *For every $p \in \{1, \ldots, n\}$ and every integer $w \in \mathbb{Z}^+ \cup \{0\}$, if $W[p]$ contains $(w, \textbf{true})$ at some time t, then $W[p]$ never contains $(w, \textbf{false})$ after time t.*
(7) *At any time t, if a $p \in \{1, \ldots, n\}$ is in the critical section, then $W[p]$ contains $(w, \textbf{true})$ for some integer $w \neq 0$.*
(8) *The CAS() operation on line 5 always succeeds.*
(9) *At any time t, if a process p reads $(w, -)$ from $W[p]$ on line 23, then $w \in \mathbb{Z}^+$.*
(10) *For all $c \in \mathbb{Z}^+$, $A[c]$ cannot be modified after it is set to a non-zero value.*
(11) *An any time t, if Token contains an integer $\ell \in \mathbb{Z}^+$, then $A[1..\ell]$ all contain non-zero values.*
(12) *For every $p \in \{1, \ldots, n\}$ and every non-negative integer w, if $W[p]$ contains $(w, -)$ immediately before p writes $(0, \textbf{false})$ to $W[p]$ on line 26 of Waiting_Room(), then $w \in \mathbb{Z}^+$ and before p does this write, p reads $(w, -)$ from $W[p]$ on line 23 and -1 from $A[w]$ on line 25 within this call of Waiting_Room().*
(13) *At any time t, for every $p \in \{1, \ldots, n\}$ and every integer $w \in \mathbb{Z}^+$, if $A[w]$ does not contain -1 and $W[p]$ is changed from $(w, \textbf{false})$, then $W[p]$ is changed to $(w, \textbf{true})$.*
(14) *At any time t, for every $p \in \{1, \ldots, n\}$ and every integer $w \in \mathbb{Z}^+$, if $A[w] = p$, then either $W[p]$ contains $(w, -)$ or $W[p]$ previously contained $(w, \textbf{true})$.*
(15) *At any time t, for every $p \in \{1, \ldots, n\}$ and every integer $w \in \mathbb{Z}^+$, if $A[w] = p$ and Token is changed from $w - 1$, then $W[p] \neq (w, \textbf{true})$ and $W[p]$ previously contained $(w, \textbf{true})$.*
(16) *At any time t, at most one entry of $W[1..n]$ contains $(-, \textbf{true})$.*
(17) *Each time line 26 is executed, there is a unique integer $w \in \mathbb{Z}^+$ such that when line 26 is executed, some entry of $W[1..n]$ is changed from containing $(w, -)$ to containing $(0, \textbf{false})$.*
(18) *At any time t, if Token contains a non-negative integer ℓ, then line 26 has been executed at most $\ell + 1$ times.*
(19) *Suppose there are only finitely many crashes. For every $p \in \{1, \ldots, n\}$ and every $w \in \mathbb{Z}^+$, if $W[p]$ contains $(w, \textbf{true})$ at some time t, then Token contains w at some time $t' > t$.*

At any time t, let C_t be the value stored in C at time t, and D_t be the value stored in D at time t. Then for every $p \in \{1, \ldots, n\}$, let $c_{p,t}$ be the greatest integer such that p has set C to $c_{p,t}$ at or before time t; or 0 if p has never modified C. Then let I_t be the set of every process p such that:

- $c_{p,t} \neq 0$.
- $A[c_{p,t}] \neq -1$ at time t.
- At time t, p has not incremented D (line 15 or 28) since setting C to $c_{p,t}$.

Lemma 2.2. *$D_t + |I_t| - C_t$ is monotonically non-decreasing throughout any interval of time when no process performs a successful CAS() on line 46.*

Lemma 2.3. *At any time t, $D_t + |I_t| - C_t \geq 0$.*

Proof. Initially, $D_0 = |I_0| = C_0 = 0$. By Lemma 2.2, $D_t + |I_t| - C_t$ is monotonically non-decreasing throughout any interval of time when no process performs a successful CAS() on line 46. So it suffices to show that at any time t, if $D_{t-1} + |I_{t-1}| - C_{t-1} \geq 0$ and a successful CAS() on line 46 is performed at time t, then $D_t + |I_t| - C_t \geq 0$.

Let $p \in \{1, \ldots, n\}$ and $\ell \in [0..\infty]$ be such that p performs a successful CAS$(0, -1)$ on $A[\ell + 1]$ on line 46 at time t. Clearly, $D_t = D_{t-1}$ and $C_t = C_{t-1}$, so if $I_{t-1} \subseteq I_t$ then $D_t + |I_t| - C_t \geq D_{t-1} + |I_{t-1}| - C_{t-1} \geq 0$ and we are done. So suppose $|I_t| < |I_{t-1}|$.

Since only a successful CAS$(0, -1)$ on $A[\ell + 1]$ is performed at time t, there must exist a process $\hat{p} \in I_{t-1}$ such that $c_{\hat{p},t} = \ell + 1$,

Shared Memory

FAI C: Initially 0.
FAI D: Initially 0.
CAS $A[1..\infty]$: Each initially 0.
CAS $W[1..n]$: Each initially (0, **false**).
CAS $Exit[1..n]$: Each initially **false**.
CAS *Token*: Initially 0.

Enter()

```
    Code for process p ∈ {1, ..., n}:
 1: procedure Enter( )
 2:   repeat
 3:     if W[p].read() = (0, false) then
 4:       c ← 1 + C.FAI()
 5:       W[p].CAS((0, false), (c, false))
 6:   until call Waiting_Room()
 7:   enter the critical section
```

Exit()

```
    Code for process p ∈ {1, ..., n}:
 8: procedure Exit( )
 9:   Exit[p].write(true)
10:   W[p].write(0, false)
11:   call Move_Token()
12:   Exit[p].write(false)
13:   return to the remainder section
```

Recover()

```
    Code for process p ∈ {1, ..., n}:
14: procedure Recover( )
15:   D.FAI()
16:   if Exit[p].read() then
17:     jump to Exit()
18:   (−, b) ← W[p].read()
19:   if b = true then
20:     enter the critical section
21:   jump to Enter()
```

Waiting_Room()

```
    Code for process p ∈ {1, ..., n}:
22: procedure Waiting_Room( )
23:   (w, −) ← W[p].read()
24:   A[w].CAS(0, p)
25:   if A[w].read() ≠ p then
26:     W[p].write(0, false)
27:     return false
28:   D.FAI()
29:   call Move_Token()
30:   repeat
31:   until W[p].read() = (w, true)
32:   return true
```

Move_Token()

```
    Code for process p ∈ {1, ..., n}:
33: procedure Move_Token( )
34:   ℓ ← Token.read()
35:   d ← D.read()
36:   c ← C.read()
37:   if ℓ < c then
38:     p′ ← A[ℓ + 1].read()
39:     if p′ > 0 then
40:       W[p′].CAS((ℓ + 1, false), (ℓ + 1, true))
41:       if W[p′].read() ≠ (ℓ + 1, true) and p = p′ then
42:         if Token.CAS(ℓ, ℓ + 1) then
43:           call Move_Token()
44:     else
45:       if c ≤ d then
46:         A[ℓ + 1].CAS(0, −1)
47:         if A[ℓ + 1].read() = −1 then
48:           if Token.CAS(ℓ, ℓ + 1) then
49:             call Move_Token()
50:   return
```

Figure 1: Pseudocode for the Recoverable Mutual Exclusion Algorithm

so $A[c_{\hat{p},t}]$ is set to -1 at time t, causing $\hat{p}$ to not be in I_t. Thus $I_t = I_{t-1} - \{\hat{p}\}$ and $|I_t| = |I_{t-1}| - 1$.

Before p executes line 46 at time t, p reads d from D on line 35, reads c from C on line 36, and then finds that $c \le d$ on line 45. Consider the time $t' < t$ when p reads c from C on line 36 i.e., $c = C_{t'}$. By Lemma 2.1 (3), D is monotonically non-decreasing, so $d \le D_{t'}$. Thus, since p finds that $C_{t'} = c \le d$, $C_{t'} \le D_{t'}$, i.e., $D_{t'} - C_{t'} \ge 0$.

We now show that $\hat{p}$ is in $I_{t'}$, so $D_{t'} + |I_{t'}| - C_{t'} \ge 1$. Recall that $c_{\hat{p},t} = \ell + 1$. By Lemma 2.1 (10), since $A[\ell+1]$ cannot be changed from a non-zero value, $A[\ell+1]$ always contains 0 before the successful CAS$(0,-1)$ on $A[\ell+1]$ at time t. Furthermore, p reads ℓ from *Token* on line 34 before time t', reads c from C on line 36 at time t', and then finds that $\ell < c$. Thus $\ell + 1 \le c = C_{t'}$. In other words, C was set to $c_{\hat{p},t} = \ell + 1$ at or before time t'. So $c_{\hat{p},t} = c_{\hat{p},t'}$. Finally, since $\hat{p} \in I_{t-1}$, $\hat{p}$ has not incremented D since setting C to $c_{\hat{p},t'}$ even at time $t-1$.

Consequently, $c_{\hat{p},t'} = c_{\hat{p},t} \neq 0$, and at time t', $A[c_{\hat{p},t'}] \neq -1$ and $\hat{p}$ has not incremented D (line 15 or 28) since setting C to $c_{\hat{p},t'}$. By definition, $\hat{p} \in I_{t'}$ as wanted. Thus $D_{t'} + |I_{t'}| - C_{t'} \ge 1$.

If no process performs a successful CAS() on line 46 within the interval $[t', t-1]$, then by Lemma 2.2, $D_{t-1} + |I_{t-1}| - C_{t-1} \ge 1$. So $D_t + |I_t| - C_t = D_{t-1} + |I_{t-1}| - 1 - C_{t-1} \ge 0$ and we would be done. Thus it suffices to show that no process performs a successful CAS() on line 46 within the interval $[t', t-1]$.

Let $p' \in \{1, \ldots, n\}$ and $\ell' \in [0..\infty]$. Suppose, for contradiction, that p' performs a successful CAS$(0,-1)$ on $A[\ell'+1]$ on line 46 within the interval $[t', t-1]$. There are three cases:

Case 1. $\ell' < \ell$. Then $\ell' + 1 \le \ell$. Recall that before p executes line 36 at time t', p reads ℓ from *Token* on line 34. So by Lemma 2.1 (11), $A[1..\ell]$ all contain non-zero values before time t'. By Lemma 2.1 (10), $A[1..\ell]$ still all contain non-zero values immediately before p' performs its successful CAS$(0,-1)$ on $A[\ell'+1]$ — contradicting that $\ell' + 1 \le \ell$.

Case 2. $\ell' = \ell$. Then by Lemma 2.1 (10), $A[\ell+1]$ would contain -1 immediately before time t — contradicting that p performs a successful CAS$(0,-1)$ on $A[\ell+1]$ at time t.

Case 3. $\ell' > \ell$. Then $\ell + 1 \le \ell'$. Before p' executes line 46, p' reads ℓ' from *Token* on line 34. So by Lemma 2.1 (11), $A[1..\ell']$ all contain non-zero values before time t'. By Lemma 2.1 (10), $A[1..\ell']$ still all contain non-zero values immediately before p performs its successful CAS$(0,-1)$ on $A[\ell+1]$ at time t — contradicting that $\ell + 1 \le \ell'$. □ Lemma 2.3

Lemma 2.4. *Suppose there are only finitely many crashes. If eventually Token = C forever, then eventually no process is ever outside the remainder section.*

Lemma 2.5. *Suppose there are only finitely many crashes. Then either Move_Token() is called infinitely many times, or eventually no process is ever outside the remainder section.*

Proof. Suppose, for contradiction, that Move_Token() is called only finitely many times, and there is no time after which no process is ever outside the remainder section. By Lemma 2.4, since there is no time after which no process is ever outside the remainder section, there is no time after which *Token* = C forever. Only lines 42 and 48 can modify *Token*. Thus since both lines are in Move_Token(), and Move_Token() is called only finitely many times, *Token* can only be changed finitely many times. By Lemma 2.1 (4), *Token* always contains a non-negative integer. By Lemma 2.1 (3), *Token* is monotonically non-decreasing. So for some non-negative integer ℓ', *Token* never contains a value greater than ℓ', and there is a time after which *Token* contains ℓ' forever. Thus by Lemma 2.1 (18), line 26 is executed at most $\ell' + 1$ times. Then, since line 26 is executed at most $\ell' + 1$ times, line 27 is also executed at most $\ell' + 1$ times.

Let t be the last time Move_Token() is called, i.e., some process $p \in \{1, \ldots, n\}$ executes line 29, 43 49, or 11 at time t.

Subclaim 2.5.1. *After time t, no process is in Recover(), Enter(), the critical section, or before line 29 in Waiting_Room(), or before line 11 in Exit().*

Proof. Suppose a process p_F is in Recover(). Then eventually p_F either crashes and starts Recover(), or starts Exit() (line 17), or enters the critical section (line 20), or starts Enter() (line 21).

Suppose a process p_F is in Enter(). Then eventually p_F either crashes and starts Recover(), or calls Waiting_Room() (line 6) or enters the critical section (line 7).

Suppose a process p_F is before line 29 in Waiting_Room(). Then eventually p_F either crashes and starts Recover(), or returns to Enter() (line 27), or calls Move_Token() (line 29). Recall that line 27 is executed at most $\ell + 1$ times, and Move_Token cannot be called after time t. So eventually p_F can only crash and start Recover(), or return to Enter() via line 32 and so break out of the repeat-until loop on lines 2-6.

Suppose a process p_F is in the critical section. Then eventually p_F either crashes and starts Recover(), or starts Exit() (line 6).

Suppose a process p_F is before line 11 in Exit(). Then eventually p_F either crashes and starts Recover(), or calls Move_Token() (line 11). Recall that Move_Token cannot be called after time t. So eventually p_F can only crash and start Recover().

Thus observe that after time t, if a process p_F is in Recover(), Enter(), the critical section, or before line 29 in Waiting_Room(), or before line 11 in Exit(), then p_F crashes infinitely often — contradicting that H only has finitely many crashes. □ Subclaim 2.5.1

By Subclaim 2.5.1, no process can crash after time t. So consider the steps taken by p after calling Move_Token() at time t. First, p reads ℓ from *Token* on line 34.

Subclaim 2.5.2. *After p reads ℓ from Token on line 34, no process ever performs a CAS($\ell, \ell + 1$) on Token (line 42 or 48).*

Proof. Suppose, for contradiction, that at least one process performs a CAS($\ell, \ell + 1$) on *Token* after p reads ℓ from *Token*. Consider the first CAS($\ell, \ell + 1$) performed on *Token* after p reads ℓ from *Token*. Since only CAS($\ell, \ell + 1$) operations (line 42 or 48) can change *Token* from ℓ, *Token* still contains ℓ immediately before the first CAS($\ell, \ell + 1$) performed on *Token* after p reads ℓ from *Token*. Thus this first CAS($\ell, \ell + 1$) succeeds.

Consider the process p_T that performs this first CAS($\ell, \ell + 1$) (line 42 or 48). Since the CAS($\ell, \ell+1$) succeeds, p_T evaluates the conditional on line 42 or 48 as **true**. Then eventually, p_T either crashes or calls Move_Token() on line 43 or 49 after time t — contradicting either Subclaim 2.5.1 or that t is the last time Move_Token() is called. □ Subclaim 2.5.2

After reading ℓ from *Token* on line 34, p reads d from D on line 35, and reads c from C on line 36. There are 3 cases:

Case 1. Process p finds that $\ell \geq c$ on line 37. Then by Lemma 2.1 (3), *Token* = C when p reads c from C on line 36. Then, since there is no time after which *Token* = C forever, eventually *Token* $\neq C$. By Lemma 2.1 (4), *Token* is always at most C, so *Token* $\neq C$ implies *Token* $< C$. By Lemma 2.1 (3), both C and *Token* are monotonically non-decreasing. Thus since *Token* $< C$ eventually, eventually some process p_C modifies C. Only line 4 of Enter() can modify C, so p_C is in Enter() after time t — contradicting Subclaim 2.5.1.

Case 2. Process p finds that $\ell < c$ on line 37, reads p' from $A[\ell+1]$ on line 38, then finds $p' > 0$ on line 39. Then by Lemma 2.1 (14), when p reads $p' \in \mathbb{Z}^+$ from $A[\ell+1]$ on line 38, either $W[p']$ contains $(\ell + 1, -)$, or $W[p']$ previously contained $(\ell + 1, \textbf{true})$.

By Lemma 2.1 (10), since p reads $p' \in \mathbb{Z}^+$ from $A[\ell + 1]$, $A[\ell + 1]$ never contains -1. By Lemma 2.1 (13), since $A[\ell + 1]$ never contains -1, if $W[p']$ is ever changed from $(\ell + 1, \textbf{false})$ to some value $(\hat{w}, \hat{b})$, then $\hat{w} = \ell + 1$ and $\hat{b} = \textbf{true}$. Consequently, either $W[p']$ contains $(\ell + 1, \textbf{false})$ forever after p reads p' from $A[\ell + 1]$ on line 38, or $W[p']$ contains $(\ell + 1, \textbf{true})$ at some time.

Since p finds $p' > 0$ on line 39, it evaluates the conditional as **true**. Recall that no process crashes after time t by Subclaim 2.5.1. So p eventually performs a CAS($(\ell+1, \textbf{false}), (\ell+1, \textbf{true})$) on $W[p']$ on line 40. Thus $W[p']$ cannot contain $(\ell + 1, \textbf{false})$ forever after p reads p' from $A[\ell + 1]$ on line 38.

So $W[p']$ contains $(\ell+1, \textbf{true})$ at some time. Then by Lemma 2.1 (19), *Token* = $\ell + 1$ at some later time. By Lemma 2.1 (3), *Token* is monotonically non-decreasing. Thus, since only lines 42 and 48 can modify *Token*, some process performs a CAS($\ell, \ell + 1$) to set *Token* to $\ell + 1$ after p reads ℓ from *Token* on line 34 — contradicting Subclaim 2.5.2.

Case 3. Process p finds that $\ell < c$ on line 37, reads p' from $A[\ell+1]$ on line 38, then finds $p' \leq 0$ on line 39. Then p eventually reaches the conditional on line 45. Suppose p it as **false**. Thus the value c that p read from C on line 36 is greater than the value d that p read from D on line 35. By Subclaim 2.5.1, no process can execute line 15 of Recover() line 4 of Enter(), or line 28 of Waiting_Room() after time t. Since these are the only lines that can modify C or D, C and D cannot be changed after time t. So after time t, C always contains c and D always contains d, so C is always greater than D.

Let t' be any time after t. By Lemma 2.3, $D_{t'} + |I_{t'}| - C_{t'} \geq 0$. Since after time t, C is always greater than D, $|I_{t'}| > 0$. So some process $\hat{p}$ is in $I_{t'}$. By definition, a process $\hat{p}$ is in $I_{t'}$ if $c_{\hat{p},t'} \neq 0$ and at time t', $A[c_{\hat{p},t'}] \neq -1$ and $\hat{p}$ has not incremented D (line 15 or 28) since setting C to $c_{\hat{p},t'}$. Thus after setting C to $c_{\hat{p},t'}$ (line 4), $\hat{p}$ has not executed line 15 or 28. Thus observe that at time $t' > t$, $\hat{p}$ must

be in RECOVER(), in ENTER(), or before line 28 in WAITING_ROOM() — contradicting Subclaim 2.5.1.

So suppose instead that p evaluates the conditional on line 45 as **true**. Recall that no process crashes after time t by Subclaim 2.5.1. So p performs a CAS$(0,-1)$ on $A[\ell+1]$, ensuring that henceforth it does not contain 0 by Lemma 2.1 (10).

Then p reaches the conditional on line 47. If p evaluates it as **true**, then p performs a CAS$(\ell,\ell+1)$ on *Token* on line 48 — contradicting Subclaim 2.5.2. So suppose p evaluates the conditional on line 47 as **false**. Thus $A[\ell+1]\neq -1$ and $A[\ell+1]\neq 0$ because p earlier performed a CAS$(0,-1)$ on $A[\ell+1]$ on line 46 and by Lemma 2.1 (10) $A[\ell+1]$ can never be changed back to 0. So after p reads $p'\leq 0$ from $A[\ell+1]$ on line 38 after time t, some process p_A changed $A[\ell+1]$ to $p_A>0$ on line 24 — contradicting Subclaim 2.5.1. □ Lemma 2.5

LEMMA 2.6. *Suppose there are only finitely many crashes. Either there is a time after which no process is ever outside the remainder section, or Token is changed infinitely often.*

LEMMA 2.7. *Suppose there are only F crashes. Then the conditional on line 3 is evaluated as **false** at most F times.*

LEMMA 2.8. *Suppose there are only finitely many crashes. Then $D_t+|I_t|-C_t$ is eventually monotonically non-increasing.*

PROOF. Since there are only finitely many crashes, there is a time t_F after which no process can crash. It suffices to show that there are only finitely many times $t>t_F$ where $D_t+|I_t|-C_t$ increases. Consider C_t. If C_t differs from C_{t-1}, then $C_t=C_{t-1}+1$ because some process p executes line 4 at time t. Consider D_t. If D_t differs from D_{t-1}, then $D_t=D_{t-1}+1$ because some process p executes line 28 at time t. Consider I_t. By definition, a process p is in I_t if $c_{p,t}\neq 0$ and at time t, $A[c_{p,t}]\neq -1$ and p has not incremented D (line 15 or 28) since setting C to $c_{p,t}$. So if I_t differs from I_{t-1}, either a process $p\notin I_{t-1}$ modifies C at time t (line 4), $A[c_{p,t}]$ for a process $p\in I_{t-1}$ is set to -1 at time t, or a process $p\in I_{t-1}$ has incremented D at time t (line 28).

It suffices to show that in all cases, barring a finite number of exceptions, $D_t+|I_t|-C_t\leq D_{t-1}+|I_{t-1}|-C_{t-1}$.

Case 1. Some process p executes line 4 at time t. Then since p increments C at time t, it is possible that $p\in I_t$ but $p\notin I_{t-1}$. Now observe that since no other process takes a step at time t, $C_t=C_{t-1}+1$, $D_t=D_{t-1}$, and either $I_t=I_{t-1}+\{p\}$ or $I_t=I_{t-1}$. So $|I_t|\leq|I_{t-1}|+1$. Thus $D_t+|I_t|-C_t\leq D_{t-1}+|I_{t-1}|+1-(C_{t-1}+1)=D_{t-1}+|I_{t-1}|-C_{t-1}$ with no exceptions.

Case 2. Some process p executes line 15 at time t. Since there are only finitely many crashes, there is a time after which no process executes line 15 of RECOVER(). Thus a process executes line 15 only finitely many times, which can all be treated as exceptions.

Case 3. Some process p executes line 28 at time t. By Lemma 2.7, since there are only finitely many crashes, the conditional on line 3 is evaluated as **false** only finitely many times. Thus, barring a finite number of exceptions, p evaluates the conditional on line 3 as **true** before calling the WAITING_ROOM (line 6) in which it executes line 28 at time t. So before calling the WAITING_ROOM, p sets C to some integer c on line 4 then set $W[p]$ to $(c,\textbf{false})$ on line 5 by Lemma 2.1 (8).

By Lemma 2.1 (5), only p can change $W[p]$ from $(c,\textbf{false})$ to $(w,-)$ for any integer $w\neq c$. So within the call of WAITING_ROOM, p reads $(c,-)$ from $W[p]$ on line 23. Then it reads p from $A[c]$ on line 25; otherwise it would not execute line 28 at time t within this execution of WAITING_ROOM(). Thus by Lemma 2.1 (10), $A[c]$ never contains -1. Finally, p increments D on line 28 at time t.

Observe that since p has not modified C again since setting C to c, $c_{p,t-1}=c$ by definition. Furthermore, after setting C to c, p does not increment D until time t. Thus, since $A[c_{p,t-1}]$ never contains -1, $p\in I_{t-1}$ by definition. Then, since p increments D at time t, $p\notin I_t$ by definition. Now observe that since no other process takes a step at time t, $|I_t|=|I_{t-1}|-1$, $D_t=D_{t-1}+1$, and $C_t=C_{t-1}$. So $D_t+|I_t|-C_t=D_{t-1}+1+|I_{t-1}|-1-C_{t-1}=D_{t-1}+|I_{t-1}|-C_{t-1}$.

Thus we have shown that, barring a finite number of exceptions, $D_t+|I_t|-C_t=D_{t-1}+|I_{t-1}|-C_{t-1}$.

Case 4. For some process $p\in I_{t-1}$, $A[c_{p,t}]$ is set to -1 at time t. Only lines 24 and 46 can modify $A[c_{p,t}]$. $A[c_{p,t}]$ cannot be set to -1 on line 24. So a successful CAS() on line 46 is performed at time t.

Observe that since no other process takes a step at time t, $D_t=D_{t-1}$, $C_t=C_{t-1}$, and every process p that was not in I_{t-1} cannot be in I_t, i.e., $|I_t|\leq|I_{t-1}|$. Thus $D_t+|I_t|-C_t\leq D_{t-1}+|I_{t-1}|-C_{t-1}$. □ Lemma 2.8

LEMMA 2.9. *For every integer $w\in\mathbb{Z}^+$, if $A[w]$ is changed from 0 to -1 at some time t, then either $D_t+|I_t|-C_t=D_{t-1}+|I_{t-1}|-C_{t-1}-1$, or the unique process that set C to w at some time $t'<t$ executes line 15 within $[t',t]$.*

LEMMA 2.10. *Suppose there are only finitely many crashes. Then only finitely many entries in $A[1..\infty]$ ever contain -1.*

We now show that the run $\mathcal{E}$ does not violate any of the following properties:

LEMMA 2.11 (MUTUAL EXCLUSION). *At any time t, there is at most one process in the critical section.*

PROOF. By Lemma 2.1 (7), if a process $p\in\{1,\ldots,n\}$ is in the critical section at time t, then $W[p]$ contains $(-,\textbf{true})$. By Lemma 2.1 (16), at most one entry of $W[1..n]$ contains $(-,\textbf{true})$ at time t. The claim immediately follows. □ Lemma 2.11

LEMMA 2.12 (WAIT-FREE RECOVERY). *If a process p begins RECOVER(), then p eventually completes RECOVER() or crashes.*

PROOF. Immediately follows from the fact that RECOVER() has no loops. □ Lemma 2.12

LEMMA 2.13 (STARVATION-FREEDOM). *Suppose there are only finitely many crashes. Then all pending method calls will terminate provided that all processes that either have pending method calls or are in the critical section keep taking steps.*

PROOF. First, by Lemma 2.12, no process can have a pending RECOVER() that never terminates.

Second, suppose, for contradiction, that that a process p has a pending EXIT() that never terminates. This is only possible if p recursively calls MOVE_TOKEN() infinitely often, i.e., p executes either line 43 or line 49 infinitely often.

Case 1. p executes line 43 infinitely often. Before each execution of line 43 by p, p successfully changes *Token* from ℓ to $\ell+1$ for

some integer ℓ on line 42. By Lemma 2.1 (3), *Token* is monotonically non-decreasing, so each uses its own unique integer ℓ.

Before p changes *Token* from ℓ to $\ell + 1$ on line 42, p finds that $p = p'$ on line 41, where p previously read p' from $A[\ell + 1]$ on line 38. By Lemma 2.1 (10), $A[\ell + 1] = p$ forever after.

Thus, there are infinitely many entries of $A[1..\infty]$ that eventually contain p forever. Then, since each entry of $A[1..\infty]$ can only be set to p when p executes line 24, p executes line 24 infinitely often — contradicting that p executes line 24 of Waiting_Room only finitely many times since p eventually becomes stuck recursively calling Move_Token() infinitely often.

Case 2. p executes line 49 infinitely often. Before each execution of line 49 by p, p successfully changes *Token* from ℓ to $\ell + 1$ for some integer ℓ on line 48. By Lemma 2.1 (3), *Token* is monotonically non-decreasing, so each uses its own unique integer ℓ.

Before p changes *Token* from ℓ to $\ell+1$ on line 48, p reads -1 from $A[\ell + 1]$ on line 39. By Lemma 2.1 (10), $A[\ell + 1] = -1$ forever after. Thus, there are infinitely many entries of $A[1..\infty]$ that eventually contain -1 forever — contradicting Lemma 2.10.

Finally, suppose, for contradiction, that that a process p has a pending Enter() that never terminates. Thus p must be stuck within some loop:

Case 1. p is stuck within the repeat-until loop on lines 2-6 forever. So p executes line 27 infinitely often. Thus p also executes line 26 infinitely often. By Lemma 2.1 (17), each time line 26 is executed, there is a unique integer $w \in \mathbb{Z}^+$ such that when line 26 is executed, some entry of $W[1..n]$ is changed from containing $(w, -)$ to containing $(0, \textbf{false})$. By Lemma 2.1 (12), before an entry of $W[1..n]$ is changed from containing $(w, -)$ on line 26, $A[w] = -1$. By Lemma 2.1 (10), $A[w]$ can never be changed from -1. Consequently, since p executes line 26 infinitely often, there are infinitely many entries in $A[1..\infty]$ that eventually contain -1 forever — contradicting Lemma 2.10.

Case 2. p is stuck within the repeat-until loop on lines 30-31 forever. Before entering the repeat-until loop on lines 30-31 forever, p reads $(w, -)$ for some integer $w \in \mathbb{Z}^+$ from $W[p]$ on line 23 by Lemma 2.1 (9). Note that by Lemma 2.1 (5), $W[p]$ henceforth can never be changed from $(w, -)$, because between reading $W[p]$ on line 23 and getting stuck forever in the repeat-until loop on lines 30-31, p can only modify $W[p]$ on line 40, which cannot change $W[p]$ from $(w, -)$.

Then p reads p from $A[w]$ on line 25; otherwise it would execute line 27 instead of reaching the repeat-until loop on lines 30-31. Note that by Lemma 2.1 (10), henceforth $A[w] = p$ forever.

Finally, p eventually becomes stuck in the repeat-until loop on lines 30-31 until it reads $(w, \textbf{true})$ from $W[p]$. Then, $W[p]$ can never be changed from $(w, -)$, p reads $(w, \textbf{false})$ from $W[p]$ on line 31 infinitely often.

By Lemma 2.6, *Token* is changed infinitely often. Initially *Token* contains 0 and *Token* is only modified by CAS() operations that increment its value (lines 42 and 48). Thus since $w \in \mathbb{Z}^+$, *Token* is eventually changed from $w - 1$ to w at some time t.

By Lemma 2.1 (11), $A[w]$ already contains a non-zero value when *Token* is changed from $w - 1$ to w. By Lemma 2.1 (10), this non-zero value must be p. So at time t, since $A[w] = p$ and *Token* is changed from $w - 1$, by Lemma 2.1 (15), $W[p] \neq (w, \textbf{true})$ and $W[p]$ previously contained $(w, \textbf{true})$. By Lemma 2.1 (6), since $W[p]$ previously contained $(w, \textbf{true})$, henceforth $W[p] \neq (w, \textbf{false})$ forever. Thus at time t, $W[p] \neq (w, \textbf{true})$ and $W[p] \neq (w, \textbf{false})$. So at time t, $W[p] = (w', -)$ for some integer $w' \neq w$ — contradicting that $W[p]$ can never be changed from $(w, -)$. □ Lemma 2.13

Lemma 2.14 (Wait-Free Critical Section Re-entry). *There is a bound b such that, if a process crashes while in the critical section, then it re-enters the critical section before completing b consecutive shared memory steps without crashing.*

Proof. Let t be the time when p crashes. Since p crashes inside the critical section, by Lemma 2.1 (2) $Exit[p] = \textbf{false}$. By Lemma 2.1 (1), only p can modify $Exit[p]$.

Furthermore, since p crashes in the critical section, by Lemma 2.1 (7), $W[p] = (w, \textbf{true})$ for some integer $w \in \mathbb{Z}^+$ at time t. By Lemma 2.1 (6), $W[p]$ cannot contain $(w, \textbf{false})$ after containing $(w, \textbf{true})$. Thus by Lemma 2.1 (5), only process p can change $W[p]$ from $(w, \textbf{true})$.

Consider the shared memory steps taken by p after time t. Within Recover(), p cannot modify $Exit[p]$ or $W[p]$. So p can only read **false** from $Exit[p]$ on line 16 Thus p cannot evaluate the conditional on line 16 as **true**. So p cannot complete Recover() on line 17. Furthermore, p can only read $(w, \textbf{true})$ from $W[p]$ on line 18. Thus p cannot evaluate the conditional on line 19 as **false**. So p can only complete Recover() by entering the critical section on line 20.

Thus it is not difficult to see that if p has taken 4 consecutive shared memory steps without crashing at some time $t' > t$, then p has entered the critical section at some time within $[t, t']$. □ Lemma 2.14

Lemma 2.15 (Wait-Free Recovery to Exit). *There is a bound b such that, if a process crashes after its first and before its last shared memory operation of Exit(), then it returns to Exit() before completing b consecutive shared memory steps without crashing.*

Proof. Let t be the time when p crashes. By Lemma 2.1 (1), only p can modify $Exit[p]$. This can only happen in lines 9 and 12. The former sets $Exit[p]$ to **true**, while the latter sets it to **false**.

Thus, when p crashes after its first and before its last shared memory operation of Exit() at time t, $Exit[p]$ = **true**. Furthermore, $Exit[p]$ will continue to contain **true** until p returns to Exit() and executes line 12 of Exit().

So consider the shared memory steps taken by p after time t. Since $Exit[p]$ continues to contain **true** until p returns to Exit(), p can only evaluate the conditional on line 16 as **true**. So p can only complete Recover() by returning to Exit() on line 17. Thus it is not difficult to see that if p has taken 3 consecutive shared memory steps without crashing at some time $t' > t$, then p has returned to Exit() at some time within $[t, t']$. □ Lemma 2.15

So the properties mutual exclusion [Lemma 2.11], wait-free recovery [Lemma 2.12], starvation-freedom [Lemma 2.13], wait-free critical section re-entry [Lemma 2.14], and wait-free recovery to exit [Lemma 2.15] are satisfied. Furthermore, deadlock-freedom follows from starvation-freedom. Finally, in the full version of this paper, we prove:

- The total RMRs incurred in any execution in which m passages begin is $O(m)$ on the DSM and the CC model.
- The worst-case RMR complexity is $O(1 + f)$ per passage for executions in which at most f process crashes occur in total.
- For executions with no crashes, FCFS is satisfied.

Thus, Theorem 2 follows.

3.4 Achieving Wait-Free Exit

Note that the Exit() method in our pseudo-code is not wait-free, because a process p may call Move_Token() in line 11, and then perform infinitely many recursive Move_Token() calls in one of lines 43 and 49. To avoid that, process p can keep track of the number of recursive Move_Token() calls it performed. Once that number increases beyond a threshold, say n^2, p simply "triggers" a Move_Token() call for each other process. In the CC model this can be achieved with a single register R that every process is monitoring while waiting in lines 30-31. In the DSM model, every process q can monitor a register $R[q]$ in its own memory segment, so to trigger a Move_Token() for each other processes, p would have to write to all registers $R[q]$, $q \neq p$. In any case, since the total number of RMRs incurred for triggering other process's Move_Token() calls is dominated by the RMRs already incurred by p during its n^2 Move_Token() calls, the asymptotic RMR complexity does not increase, but p's Exit() method becomes wait-free.

4 CONCLUSION

This paper presents the first recoverable mutual exclusion algorithm that achieves constant amortized RMR complexity without relying on primitives that common hardware architectures do not provide. Our algorithm uses an approach that is very different from earlier algorithms, which are inspired by standard queue locks [3, 13] and rely on FAS objects. While FAS objects are the synchronization primitives of choice for non-recoverable mutual exclusion, we believe that they are not strong enough to obtain constant RMR complexity in the crash-recovery model.

Our algorithm uses unbounded space, but we believe that it is possible to apply standard memory reclamation techniques to obtain a bounded space solution even in the absence of garbage collection. Instead of adding ad-hoc memory reclamation to our algorithm, which would distract from the core ideas of solving mutual exclusion, we believe that general memory reclamation techniques for the crash-recovery model should be studied.

We also believe that it is possible to achieve the strong starvation-freedom property of [11], without sacrificing constant amortized RMR complexity. In our algorithm, a process can only starve without crashing infinitely many times, if its ticket gets skipped repeatedly. However, if that happens more than, say, $\log n$ times, then the process might be able to switch to a different mutual exclusion algorithm with $O(\log n)$ RMR complexity. Making the algorithm first-come first-served seems more difficult.

Probably the most interesting open problem is to determine whether there is a recoverable mutual exclusion algorithm that incurs only constant RMRs per passage in the (non-amortized) worst-case, even if infinitely many crashes can occur.

ACKNOWLEDGMENTS

We thank the anonymous reviewers for their valuable comments. Support is gratefully acknowledged from the Natural Science and Engineering Research Council of Canada (NSERC) under Discovery Grant RGPIN-2019-04852 and the Canada Research Chairs program.

REFERENCES

[1] Zahra Aghazadeh and Philipp Woelfel. 2016. Upper Bounds for Boundless Tagging with Bounded Objects. In *Proc. of 30th DISC*. 442–457.
[2] Hagit Attiya, Danny Hendler, and Philipp Woelfel. 2008. Tight RMR Lower Bounds for Mutual Exclusion and Other Problems. In *Proc. of 40th ACM STOC*. 217–226.
[3] Travis Craig. 1993. Building FIFO and Priority-Queuing Spin Locks from Atomic Swap. Technical Report TR-93-02-02, Department of Computer Science, University of Washington.
[4] Sahil Dhoked and Neeraj Mittal. 2020. An Adaptive Approach to Recoverable Mutual Exclusion.. In *Proc. of the 2020 ACM PODC*. To appear.
[5] Rotem Dvir and Gadi Taubenfeld. 2017. Mutual Exclusion Algorithms with Constant RMR Complexity and Wait-Free Exit Code. In *Proc. of 21st OPODIS*. 17:1–17:16.
[6] Wojciech Golab, Vassos Hadzilacos, Danny Hendler, and Philipp Woelfel. 2012. RMR-Efficient Implementations of Comparison Primitives Using Read and Write Operations. *Distr. Comp.* 25, 2 (2012), 109–162.
[7] Wojciech Golab and Danny Hendler. 2017. Recoverable Mutual Exclusion in Sub-logarithmic Time. In *Proc. of the 2017 ACM PODC*. ACM, 211–220.
[8] Wojciech Golab and Danny Hendler. 2018. Recoverable Mutual Exclusion Under System-Wide Failures. In *Proc. of the 2018 ACM PODC*. ACM, 17–26.
[9] Wojciech Golab and Aditya Ramaraju. 2019. Recoverable mutual exclusion. *Distr. Comp.* 32, 6 (2019), 535–564.
[10] Prasad Jayanti, Siddhartha Jayanti, and Anup Joshi. 2018. Optimal Recoverable Mutual Exclusion Using only FASAS. In *Proc. of 6th NETYS (LNCS)*, Vol. 11028. Springer, 191–206.
[11] Prasad Jayanti, Siddhartha V. Jayanti, and Anup Joshi. 2019. A Recoverable Mutex Algorithm with Sub-logarithmic RMR on Both CC and DSM. In *Proc. of the 2019 ACM PODC*. ACM, 177–186.
[12] Prasad Jayanti and Anup Joshi. 2017. Recoverable FCFS Mutual Exclusion with Wait-Free Recovery. In *Proc. of 31st DISC (LIPIcs)*, Vol. 91. 30:1–30:15.
[13] John M. Mellor-Crummey and Michael L. Scott. 1991. Algorithms for Scalable Synchronization on Shared-Memory Multiprocessors. *ACM Trans. Comput. Syst.* 9, 1 (1991), 21–65.
[14] Maged M. Michael. 2004. Hazard Pointers: Safe Memory Reclamation for Lock-Free Objects. *IEEE Trans. Parallel Distrib. Syst.* 15, 6 (2004), 491–504.
[15] Aditya Ramaraju. 2015. *RGLock: Recoverable mutual exclusion for non-volatile main memorysystems.* Master's thesis. University of Waterloo. https://uwspace.uwaterloo.ca/handle/10012/9473
[16] Jae-Heon Yang and James H. Anderson. 1995. A Fast, Scalable Mutual Exclusion Algorithm. *Distr. Comp.* 9, 1 (1995), 51–60.